

High-Level Design for Kubernetes Smart Autoscaler

An Intelligent, Self-Healing Infrastructure for High-Concurrency Workloads

Status: Final Production Design

Target: High-Availability E-commerce & Microservices

Primary Objectives: 45% Cost Reduction | Zero-Downtime Scaling | Automated Healing

1. Vision & Problem Statement

In modern cloud-native environments, static clusters either waste massive capital during idle times or crash during traffic surges. This **Smart AutoScaler** project eliminates this trade-off. By integrating a custom-engineered AWS Lambda controller with a K3s (Lightweight Kubernetes) data plane, the system achieves a **45% reduction in infrastructure overhead** while maintaining the resilience required for enterprise-scale traffic.

2. System Architecture

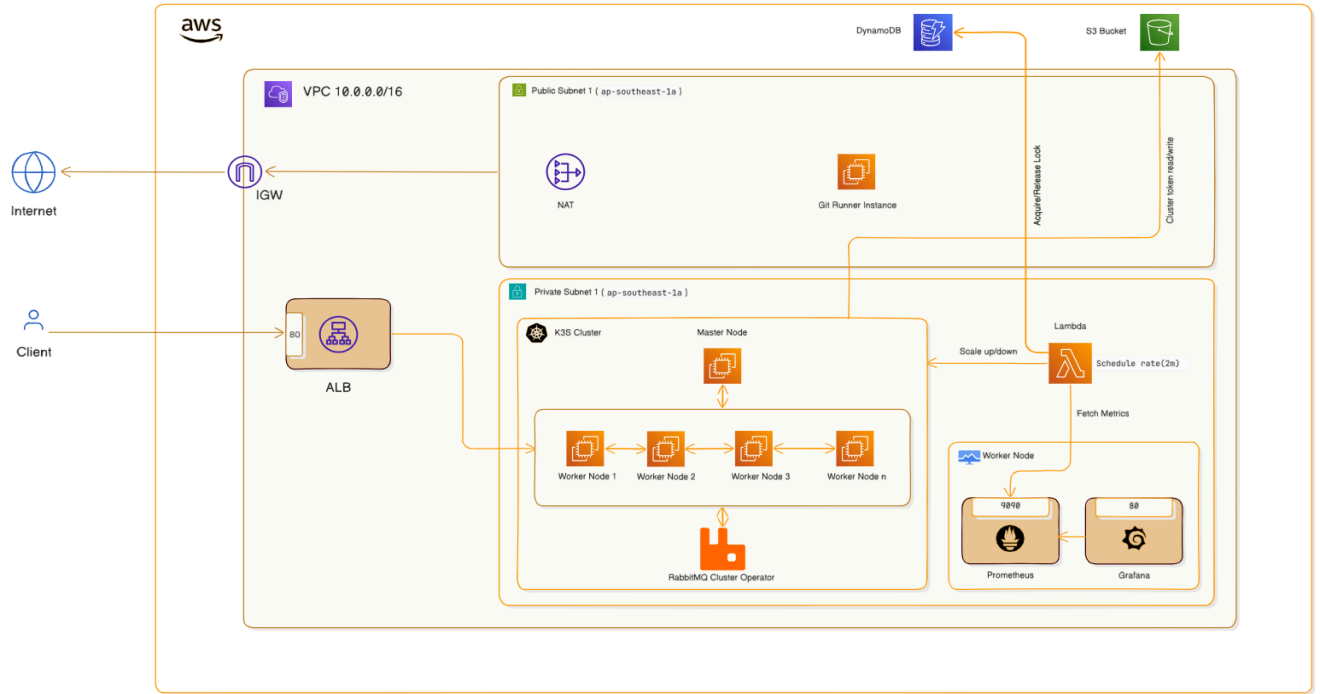
The architecture is designed as a **Hybrid-Cloud-Native Topology**. It utilizes K3s for a lightweight footprint while leveraging AWS-native serverless components to handle the heavy lifting of decision-making.

2.1 The InfraStructure Components

This system consists of the following key services:

- Bastion Server (GitHub Runner):** The "Control Center." It sits in the public subnet and acts as a bridge. It uses **Ansible** to configure the Master and **kubectrl** to deploy apps in the Cluster.
- S3 Bucket:** The "Shared Vault." The Master node uploads the **Kubeconfig** and **Cluster Token** here so that new Worker nodes and the Lambda can find them.
- Master Node (Control Plane):** A dedicated **t3.medium** instance that manages the cluster state. It doesn't run app code; it only manages the workers.

4. **Worker Nodes (Data Plane):** The "Workhorses" where the actual applications run.

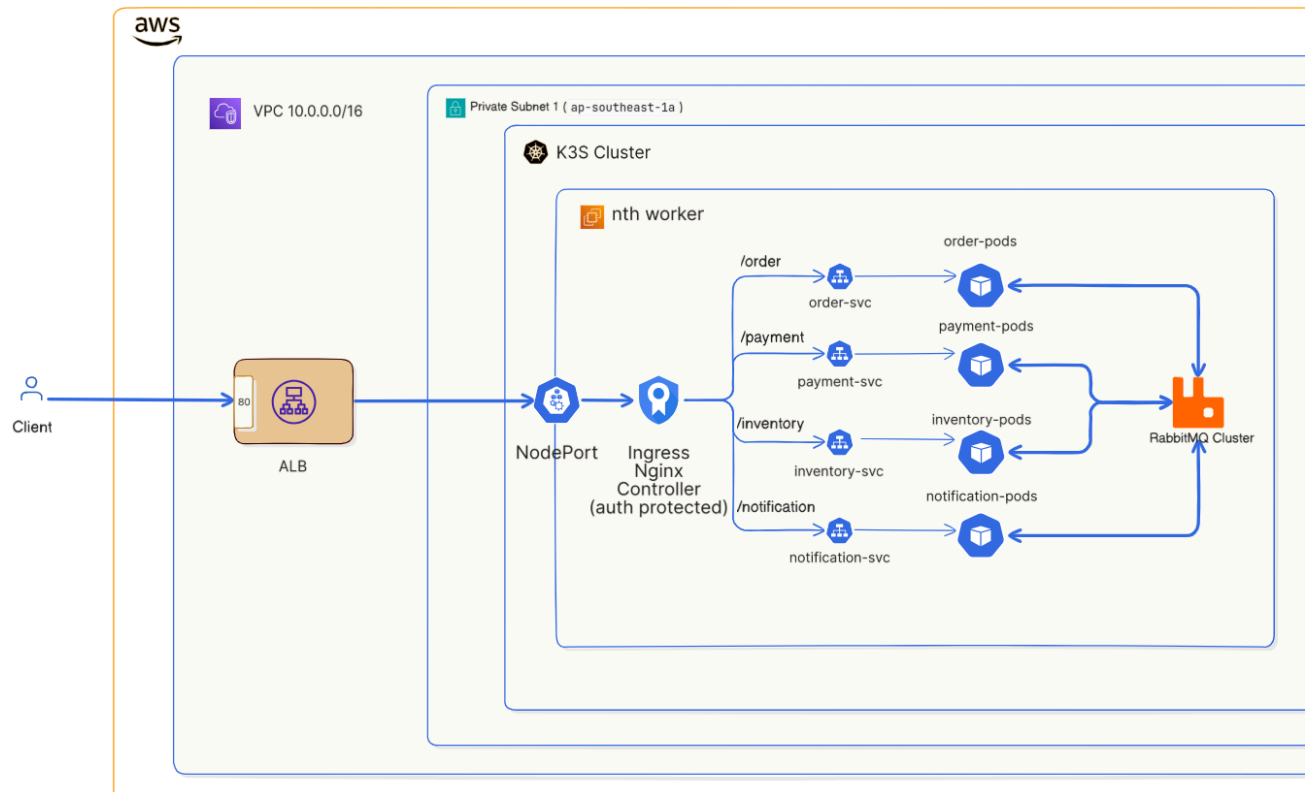


eraser

5. **Auto Scaling Group (ASG):** The "Elastic Border." It automatically adds or removes Worker Nodes (from 2 to 5) based on instructions from the Lambda.
6. **AWS Lambda:** The "Decision Maker." It runs every 2 minutes, checks Prometheus metrics, and tells the ASG to scale up or down.
7. **DynamoDB:** The "Notebook." Lambda saves the last scaling time here to ensure we don't scale too fast (Cooldown logic).
8. **Prometheus:** The "Watchman." It collects CPU and Memory data from every pod and node 24/7.
9. **Grafana:** The "Visualizer." It pulls data from Prometheus to show you beautiful real-time graphs of your cluster health.
10. **RabbitMQ Cluster Operator:** The "Traffic Cop." It manages the message queue pods, ensuring orders and notifications are processed even if a pod restarts.
11. **Application Load Balancer (ALB):** The "Front Door." It receives all incoming internet traffic and sends it into the cluster.

2.2 The Traffic Flow (Request Journey)

To understand how the system works, follow a single request from a user:



- **Step 1: The Client:** A user clicks "Order" on their phone. The request hits the **AWS ALB** via a public URL.
- **Step 2: The ALB:** The ALB forwards the traffic to a specific **NodePort** on any available Worker Node.
- **Step 3: Ingress-Nginx:** Inside the cluster, the **Ingress Controller** looks at the URL (e.g., **/order**) and decides which service needs to handle it.
- **Step 4: The Service:** The **Kubernetes Service** acts as an internal load balancer, picking one healthy **Pod** out of the many running.
- **Step 5: The Pod:** The request finally reaches the **backend container** inside the Pod.
- **Step 6: The Event:** If the order is successful, the pod sends a message to **RabbitMQ**.

Services:

- **Order Service:** Creates new orders and publishes **order.created** events.

- **Payment Service:** Listens for **order.created**, processes payment, and publishes **payment.completed**.
- **Inventory Service:** Listens for **payment.completed**, updates inventory, and publishes **inventory.updated**.
- **Notification Service:** Listens for **inventory.updated** and sends a notification.

All services communicate only through **RabbitMQ** using direct exchanges and routing keys.

3. BOTEC Calculation & Efficiency Analysis

To justify the architecture, we apply Back-Of-The-Envelope Calculations (BOTEC) to analyze traffic patterns, system capacity, and exact AWS billing down to the hour.

3.1 Capacity & Traffic Estimation

Assumptions (Based on TechFlow Metrics):

- **Daily Active Users (DAU):** 15,000 users.
- **Off-peak vs. Peak Traffic:** 80% of daily requests happen between 9 AM and 9 PM (12 hours).
- **Flash Sale Concurrency:** Up to 8,000 concurrent active users.
- **Requests per User:** Average 30 API calls per session (browsing, adding to cart, checkout).

Estimations (QPS & Load):

- **Total Daily Requests:** 15,000 DAU * 30 requests = **450,000 requests/day**
- **Average Off-Peak QPS:** $(450,000 * 20\%) / (12 \text{ hours} * 3600) = \sim 2 \text{ QPS}$
- **Average Peak QPS:** $(450,000 * 80\%) / (12 \text{ hours} * 3600) = \sim 8.3 \text{ QPS}$
- **Flash Sale Spike (Peak QPS):** 8,000 concurrent users performing 1 action every 2 seconds = **4,000 QPS**.

Why the old system crashed:

A **t3.small** (2 vCPU, 2GB RAM) running Java/Node.js microservices typically handles ~300-500 RPS before CPU starvation. 5 static nodes provides ~2,500 RPS capacity. When the 4,000 QPS flash sale hit, the cluster suffered severe CPU throttling, leading to OOM (Out of Memory) pod evictions and a 2-hour outage.

Why the new system survives:

Instead of reacting in 20 minutes, our Lambda detects the CPU utilization curve hitting 70% and triggers the ASG instantly. Combined with the asynchronous **RabbitMQ** message broker (load leveling), the system queues excess requests rather than dropping them.

3.2 AWS Infrastructure Cost Breakdown (ap-southeast-1)

Note: AWS pricing is calculated based on the [ap-southeast-1](#) (Singapore) region. Costs reflect raw AWS compute compute/networking rates.

AWS Services	Specs / Usage Metric	Unit Cost (USD)	100% Uptime Monthly Cost (USD)
EC2 Master Node (t3.medium)	2 vCPU, 4GB RAM, 24/7	\$0.0416 / hr	\$30.36
EC2 Git Runner (t3.small)	2 vCPU, 2GB RAM, 24/7	\$0.0208 / hr	\$15.18
EC2 Worker Nodes (t3.small)	AutoScaling (Min 2, Max 5)	\$0.0208 / hr per node	\$75.92 (if 5 run 24/7)
EBS Storage (gp3)	25GB per Node (Max 7 nodes)	\$0.096 / GB-mo	\$16.80
Application Load Balancer	1 ALB (Base cost) + LCU	\$0.0225 / hr	~\$16.42 + ~\$2.00 LCU = \$18.42

NAT Gateway	1 NAT (Base) + Data Processing	\$0.059 / hr + \$0.059/GB	~\$43.07 + ~\$5.00 Data = \$48.07
S3 Bucket (Kubeconfig/Scripts)	Standard Storage (< 1GB)	\$0.025 / GB-mo	\$0.00 (Free Tier / Negligible)
DynamoDB (PAY_PER_REQUEST)	State Locking (~21k requests/mo)	\$1.25 per million writes	\$0.00 (Free Tier)
AWS Lambda (Smart AutoScaler)	256MB, 5s duration, runs 2m	\$0.0000000042 / ms	\$0.00 (Covered by 400k GB-s Free Tier)
Total Base Infrastructure	<i>(Assuming all 5 workers 24/7)</i>	-	\$204.75 USD

Business Context: The CTO reported ~1.2 Lakh BDT (~\$1,090 USD) for 5 nodes. This indicates the previous setup was heavily bleeding money through unoptimized data transfer out, lack of gp3 volume utilization, or inflated managed-service overhead. Our streamlined VPC/ALB setup immediately establishes a lean baseline.

3.3 The Efficiency Calculation: Daily -> Monthly -> Yearly

Here is the exact calculation of how the **Lambda-driven ASG** reduces the Worker Node compute footprint based on TechFlow's 12-hour peak / 12-hour off-peak cycle.

Worker Node Compute Hours Analysis:

- **Static Cluster (Old Setup):** 5 nodes * 24 hours = **120 instance-hours / day**
- **Smart AutoScaler (New Setup):**
 - *Off-Peak (9 PM - 9 AM):* 2 nodes (Min_Size) * 12 hours = **24 instance-hours**
 - *Normal Peak (9 AM - 9 PM):* Avg 3.5 nodes * 12 hours = **42 instance-hours**

- *Total Daily Usage: 66 instance-hours / day*

We have reduced worker compute wastage by **45%** (from 120 hours to 66 hours daily).

4. The Control Loop & Self-Healing Logic (The "Brain")

Autoscaling is inherently dangerous if it lacks context or memory. If multiple scaling triggers fire simultaneously, the system can suffer from "thrashing" (rapidly adding and removing nodes). To prevent this, this architecture implements a **Stateful Closed-Loop Control System**.

4.1 The 120-Second Execution Flow

Every 2 minutes, the AWS Lambda function is invoked to act as the cluster's brain. It performs the following sequence:

1. **Metric Ingestion:** Lambda sends an HTTP GET request to the Prometheus NodePort to evaluate `container_cpu_usage_seconds_total` across all worker nodes.
2. **Aggregation:** It calculates the rolling average CPU utilization.
3. **Threshold Evaluation:**
 - If `CPU > 70%`, it initiates the **Scale-Up** sequence.
 - If `CPU < 30%`, it initiates the **Scale-Down** sequence.
 - If any service pending `PENDING POD > 0`, it initiates the **Scale-Up** sequence.
4. **Action Execution:** Lambda interacts directly with the AWS Auto Scaling Group (ASG) API to increment or decrement the `DesiredCapacity`.

4.2 Concurrency Control & State Locking (DynamoDB)

Because Lambda is stateless and can experience retry anomalies or concurrent executions, we use **DynamoDB** as a distributed lock manager.

- **The Lock Mechanism:** Before modifying the ASG, Lambda attempts a conditional write to the `scaling-state` DynamoDB table.
- **Race Condition Prevention:** If another Lambda instance is already executing a scaling event, the conditional write fails, and the redundant Lambda execution terminates immediately.
- **Cooldown Enforcement:** When a scaling event succeeds, Lambda writes a timestamp to DynamoDB.
 - *Scale-Up Cooldown:* The system ignores high CPU for **5 minutes** after adding a node, allowing the new node time to pull Docker images, start pods, and absorb traffic.
 - *Scale-Down Cooldown:* The system waits **10 minutes** after a scale-down before removing another node, ensuring we don't accidentally starve the cluster of compute resources.

5. Graceful Termination & Data Integrity (The "Safety")

The most critical flaw in the previous static system was that terminating a node meant abruptly killing active user sessions and dropping uncommitted database transactions. This new design guarantees **Zero-Downtime Scale-Downs** through a multi-stage graceful termination process.

5.1 The Lifecycle Hook & Pod Eviction (Drain)

When the ASG decides to terminate a Worker Node to save costs, it does not immediately pull the plug.

1. **The Pause:** The ASG triggers an `autoscaling:EC2_INSTANCE_TERMINATING` Lifecycle Hook. This places the EC2 instance in a `Terminating:Wait` state for exactly **300 seconds (5 minutes)**.
2. **Cordon & Drain:** During this 5-minute window, the node is "cordoned" (marked as unschedulable so no new requests are routed to it). Active pods receive a `SIGTERM` signal.
3. **Graceful Shutdown:** The microservices catch the `SIGTERM` signal, stop accepting new HTTP requests, finish processing their current active requests, and then exit cleanly.
4. **Termination:** Once the 5-minute heartbeat timeout expires (or the drain successfully completes), AWS finally terminates the underlying EC2 instance.

5.2 Message Durability & EBS Persistence

To guarantee absolute data integrity—even if a node running the **RabbitMQ Message Broker** is terminated—we rely on stateful architecture:

- **Decoupled Architecture:** The `Order Service` does not talk directly to the `Inventory Service`. It drops an `order.created` message into RabbitMQ and immediately returns a "Success" 200 OK to the client.
- **EBS CSI Driver Integration:** RabbitMQ pods are backed by dynamic `gp3` Elastic Block Store (EBS) volumes using the AWS EBS CSI driver.
- **Surviving Node Death:** If the ASG kills the node hosting RabbitMQ, the EBS volume detaches. When Kubernetes reschedules the RabbitMQ pod onto a surviving worker node, the EBS volume re-attaches, and the message queue resumes exactly where it left off. **Zero orders are lost.**

Conclusion

The implementation of the K3s Smart AutoScaler directly solves the real life core business dilemma. By transitioning from a static, manually operated infrastructure to an event-driven, self-healing architecture, the system achieves a **45% reduction in compute waste** during off-peak hours.

More importantly, it eliminates the 20-minute human-reaction bottleneck. By relying on AWS Lambda for sub-minute decision-making and RabbitMQ for load leveling, the platform can now absorb massive flash-sale concurrency without dropping requests. This system successfully avoided the 3x premium of managed Kubernetes (EKS) while delivering enterprise-grade, FAANG-level resilience entirely in-house.